

Fakenix

Este projeto tem como objetivo demonstrar o processo de boot de um computador que usa o firmware BIOS e noções básicas de como os recursos do computador são gerenciados em **modo real**.

O "modo real" (real mode) é a forma mais básica de operação das CPUs na arquitetura x86. Surgiu nos primeiros processadores Intel 8086.

As principais características do modo real são:

- A CPU trabalha com endereços de 20 bits, permitindo acessar até 1 MB de memória RAM (mesmo em sistemas com muito mais memória).
- Todos os programas podem acessar qualquer parte da memória. Não há isolamento entre processos.
- Só um programa pode ser executado por vez. Não há multitarefa nativa.

O esquema de endereçamento de memória em modo real é baseado em segmentos, cada segmento tendo 65.536 bytes. Existem, no total, seis registradores de segmento na arquitetura x86, cada um com 16 bits, alguns deles com funções bem específicas. São eles:

CS (Code Segment): Aponta para o segmento do código executável.

DS (Data Segment): Aponta para o segmento onde os dados estão armazenados.

SS (Stack Segment): Aponta para o segmento da pilha.

ES (Extra Segment): Segmento adicional para dados.

FS (Extra Segment): Segmento adicional para dados.

GS (Extra Segment): Segmento adicional para dados.

O endereço do segmento em memória física em modo real é calculado aplicando a equação:

$$\text{Endereço Físico} = \text{Segmento} * 16 + \text{Offset}$$

Por exemplo, se precisa-se acessar o endereço físico de memória com índice 0x9000 usando o registrador de segmento SS, calcula-se da seguinte forma:

$$SS = 0x900 * 16 + 0x0$$

1. Boot

Quando se liga um computador que usa o firmware BIOS, a inicialização do sistema se dá nas seguintes etapas:

1. A fonte de energia interna é ligada e inicia. A fonte leva algum tempo até ser capaz de gerar energia estável para o resto do computador. Então o chipset gera um sinal de reset para o processador até que ele receba um sinal de energia estável da fonte (Power Good).

- 2.Quando o sinal de reset cessa, o processador está pronto para executar. O processador é pré-programado para acessar primeiramente a ROM e executar o BIOS.
- 3.O BIOS executa o POST (Power On Self Test). Caso ocorra algum erro fatal em algum teste do POST, como a ausência de um item de hardware obrigatório, o processo de BOOT pára e é emitido um código de erro, geralmente em forma de luz ou sinais sonoros.

- 4.O BIOS executa outros BIOS de outros dispositivos (ex., placa de vídeo, HD SCSI).
- 5.Caso o BIOS suporte o padrão Plug and Play, ele detecta e configura todos os dispositivos Plug and Play.
- 6.O BIOS exibe por um curto intervalo de tempo uma tela de sumário mostrando a configuração do sistema.

7.O BIOS procura o dispositivo de BOOT, de acordo com a seqüência de BOOT armazenada no CMOS. CMOS é uma pequena memória que armazena informações de configuração de hardware.

8.Tendo identificado o dispositivo de BOOT, o BIOS procura pelas informações de BOOT, para que o carregamento do sistema operacional possa começar.

9.O BIOS procura pelo MBR (Master Boot Record) no cilindro zero, cabeça zero e setor um do dispositivo (primeiro setor do disco). Caso o BOOT seja pela USB, ele também procura pelo volume BOOT (similar ao MBR do HD).

10.O BIOS inicia o processo de carregamento do sistema operacional para a memória RAM utilizando as informações contidas no MBR. A partir deste ponto as instruções contidas neste setor do disco passam a comandar o PC.

11.Caso nenhuma informação de Boot seja encontrada no setor do primeiro dispositivo da sequência de BOOT, tenta-se o segundo dispositivo, e assim sucessivamente, até que as instruções para o carregamento do sistema operacional sejam encontradas.

2. Bootloader

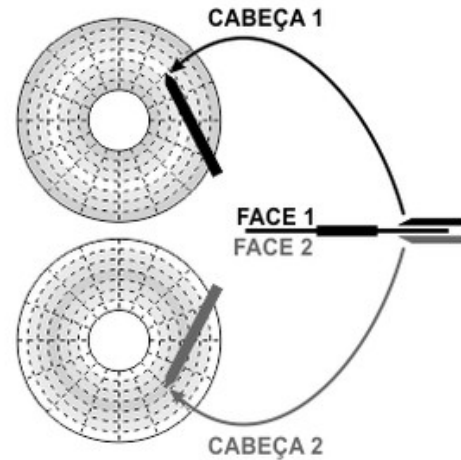
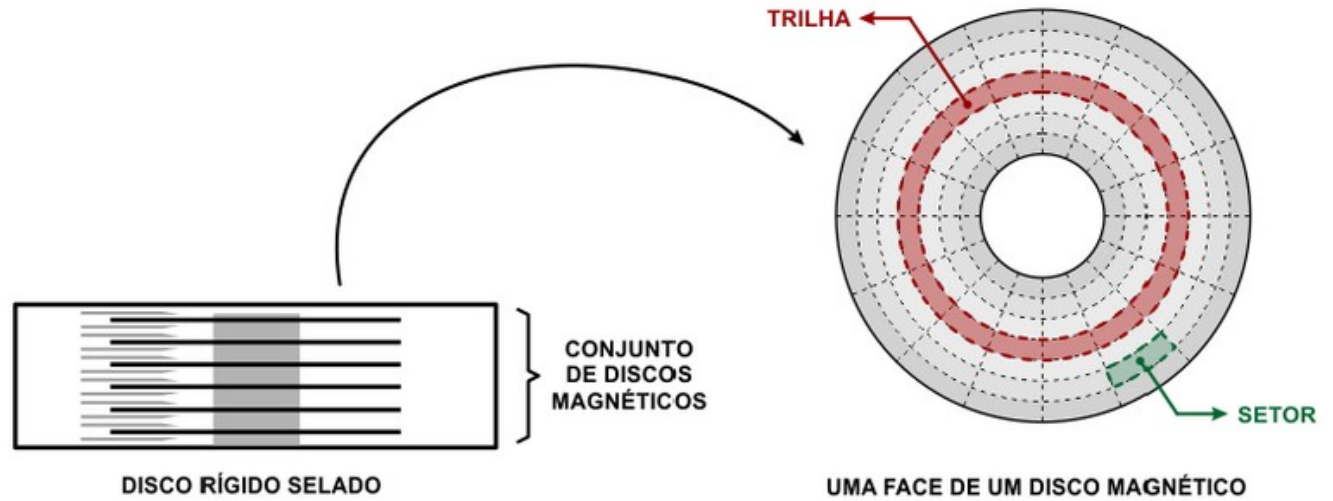
Na etapa 9 da seção anterior, o BIOS procura por um código localizado num setor específico do disco, que é chamado de MBR (Master Boot Record), localizado no cilindro zero, cabeça zero e setor um (o primeiro setor do disco). Este programa é chamado de **bootloader**. A função do bootloader é carregar na memória RAM o kernel do sistema operacional.

Como o código no setor MBR contém apenas 512 bytes, normalmente o bootloader é dividido em múltiplos estágios (multi-stage bootloader), sendo o primeiro deles o que está gravado no MBR.

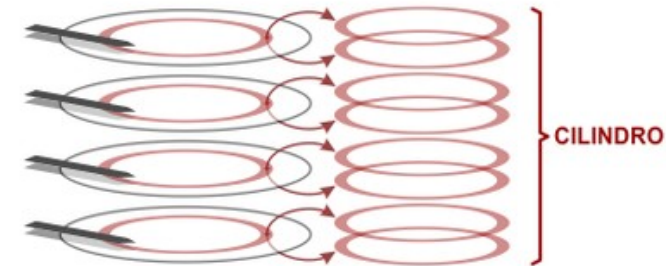
Figura 1. Geometria do disco rígido (HD)

A formatação física de um disco rígido, feita pelo fabricante, define as trilhas, setores e cilindros.

Alguns HD's tem múltiplos discos, cada disco tendo duas cabeças de leitura e escrita, uma para cada face.



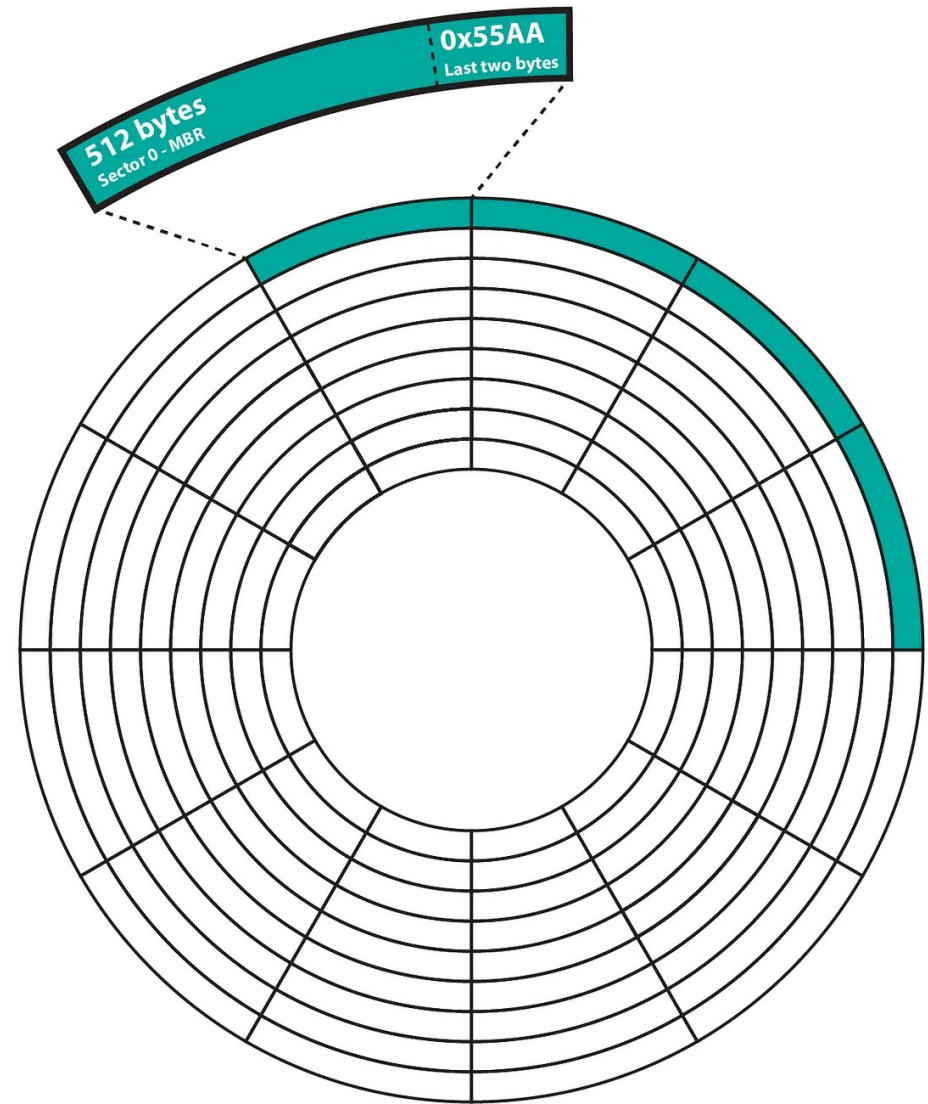
UMA CABEÇA PARA CADA FACE DE UM DISCO



CILINDRO: CONJUNTO DE TRILHAS SUPERPOSTAS

Figura 2. Bootloader multiestágios

O setor MBR (em ampliação) é o primeiro setor do disco, e contém somente 512 bytes. Geralmente não é suficiente para carregar o kernel do sistema operacional. Logo, é comum que o bootloader tenha dois ou mais estágios, ocupando mais do que um setor do disco (destaque em verde no diagrama).



Os estágios do bootloader podem estar em setores contíguos ao MBR, como mostrado na Figura 2, mas isto não é obrigatório. Caso não estejam contíguos, o bootloader deve saber como acessar o sistema de arquivos, ou deve conter uma tabela de LBA (endereços de setor), indicando aonde cada “pedaço” do segundo estágio está.

O MBR contém algumas estruturas de dados que reduzem o número de bytes disponíveis para instruções de máquina a menos de 512 bytes. Estas estruturas incluem BPB/EBPB (BIOS parameter block/Extended BIOS parameter block), que descreve o layout físico de um volume de armazenamento de dados, tabela de partições (partitions table), Id do disco (disk id).

Além disso, o MBR precisa também de uma assinatura (magic number), para indicar para o BIOS que aquele é um dispositivo inicializável, correspondendo aos valores **0x55** e **0xAA** nos bytes 511 e 512, respectivamente.

Todo o código do executável que vai carregar o sistema operacional deve caber nos bytes restantes.

O carregamento do bootloader para a memória RAM acontece na etapa seguinte ao POST. O BIOS lê os 512 bytes do MBR e carrega no endereço **0x7C00** da memória RAM. Feito isso, entrega o controle do computador para o programa, que geralmente tem a função de carregar o segundo estágio na memória. Este, sem a restrição de ter apenas 512 bytes, pode realizar as etapas de troca para o modo protegido e carregamento do kernel do sistema operacional, que então assume o controle para carregar os demais programas necessários.

Para entender melhor este processo, antes é preciso entender como a memória RAM estará organizada no momento em que o primeiro estágio do bootloader é carregado no endereço 0x7C00.

Os endereços anteriores á 0x7C00 estão parcialmente alocados para o BIOS executar as suas funções.

O espaço que vai do endereço 0x0 até o 0x3FF está alocado para a **tabela de vetores de interrupção (Interruption Vector Table – IVT)**.

A IVT é essencialmente um array de 256 entradas. Cada entrada (numeradas de 0 a 255) contém um ponteiro de 4 bytes que especifica o endereço da **rotina de tratamento de interrupção** (Interruption Service Routine - ISR) para aquele vetor. Esse ponteiro é composto por:

- ◆ Segmento (2 bytes): Indica o segmento de memória onde a ISR reside.
- ◆ Offset (2 bytes): Indica o deslocamento dentro desse segmento onde a ISR começa.

Como cada entrada da IVT ocupa 4 bytes, e existem 256 entradas, o tamanho total da tabela é de $256 \times 4 = 1024$ bytes (1 KB).

O BIOS inicializa a IVT somente com as rotinas básicas para permitir a funcionalidade essencial durante o boot. Posteriormente, o sistema operacional alterará esta tabela quando ele for carregado.

Exemplo de alocação de memória pela IVT:

Endereço (Hex)	Conteúdo (Bytes)	Descrição
00000 - 00003	Endereço da ISR para INT 00h	Divisão por zero
00004 - 00007	Endereço da ISR para INT 01h	Single-step (depuração)
00008 - 0000B	Endereço da ISR para INT 02h	Non-maskable interrupt (NMI) - falha de hardware grave
0000C - 0000F	Endereço da ISR para INT 03h	Breakpoint
00010 - 00013	Endereço da ISR para INT 04h	Overflow
...	...	...
00020 - 00023	Endereço da ISR para INT 08h	Rotina de tratamento do RTC (Real-Time Clock)
00024 - 00027	Endereço da ISR para INT 09h	Rotina de tratamento do Teclado
...	...	...
00078 - 0007B	Endereço da ISR para INT 1Eh	Rotina de tratamento de falha de disco
...	...	...
000F0 - 000F3	Endereço da ISR para INT 3Ch	Rotina de tratamento para o mouse (geralmente)
...	...	...
003FC - 003FF	Endereço da ISR para INT FFh	(Reservado)

No projeto, duas entradas na IVT serão alteradas: A entrada do temporizador do sistema (PIT), que corresponde à interrupção 0x8; e a de teclado, que corresponde à interrupção 0x9. Vão ser modificadas as ISR's padrão delas para tratar eventos do jogo da cobrinha, que será carregado no lugar de um kernel.

O temporizador será utilizado para atualizar a posição da cobrinha. O teclado muda a direção da cobrinha e também trata a reinicialização do jogo, velocidade da cobrinha e desligamento do computador.

Na sequência ao último byte da IVT, do endereço 0x400 até o 0x4FF está a **área de dados do BIOS (BIOS Data Area – BDA)**. Esta é uma área de 256 bytes que o BIOS usa para armazenar configurações cruciais de hardware, informações de status e dados temporários durante a operação do sistema, especialmente antes que o sistema operacional seja carregado.

Algumas das principais informações armazenadas na BDA incluem:

Endereços das portas de comunicação: Endereços base de porta de E/S para as portas seriais (COM) e paralelas (LPT).

Lista de equipamentos instalados: Flags (indicadores) que sinalizam o hardware presente no sistema, como o número de unidades de disco, a presença de um coprocessador matemático, o tipo de tela, etc.

Tamanho da memória: A quantidade de memória convencional e estendida disponível no sistema.

Buffer do teclado: Um pequeno buffer que armazena as teclas digitadas antes que sejam processadas pelo sistema operacional.

Modo de exibição: O modo de vídeo atual (por exemplo, modo texto ou gráfico, resolução).

Dados do relógio em tempo real (RTC): Informações relacionadas ao relógio do sistema.

Informações de Boot: Flags de status e parâmetros usados durante o processo de inicialização.

Status das unidades de disco: Informações sobre as unidades de disco conectadas.

O BIOS inicializa o BDA durante o Teste Automático de Inicialização (POST) e vai atualizando conforme necessário. Uma vez que o sistema operacional assume o controle, ele também pode ler ou usar algumas das informações armazenadas na BDA.

Acima do BDA, do endereço 0x500 até 0x7BFF é um espaço de memória que não está alocado. Pode, inclusive, com muito cuidado, ser utilizado como a pilha do bootloader.

Do endereço 0x7C00 até o 0x7DFF é o espaço reservado para o primeiro estágio do bootloader, carregado do MBR.

Do endereço 0x7E00 até 0x9FBFF corresponde à área livre de memória normalmente alocada para carregar os demais estágios do bootloader e o kernel do sistema operacional.

Do endereço 0x9FC00 até 0x9FFFF se encontra a **área de dados do BIOS estendida (Extended BIOS Data Area - EBDA)**. Sua principal finalidade é fornecer um espaço adicional para o BIOS armazenar dados específicos do sistema e de dispositivos, além da área de dados do BIOS (BDA) tradicional, que possui apenas 256 bytes.

Do endereço 0xA0000 até 0xBFFFF se encontra a **memória de vídeo (Video Memory)**.

Essa faixa é dividida para diferentes propósitos relacionados ao vídeo:

0xA0000 - 0xAFFFF (64 KB): Geralmente reservado para a memória da placa de vídeo no modo gráfico EGA/VGA (Enhanced Graphics Adapter/Video Graphics Array). É aqui que os dados dos pixels que estão sendo exibidos são armazenados.

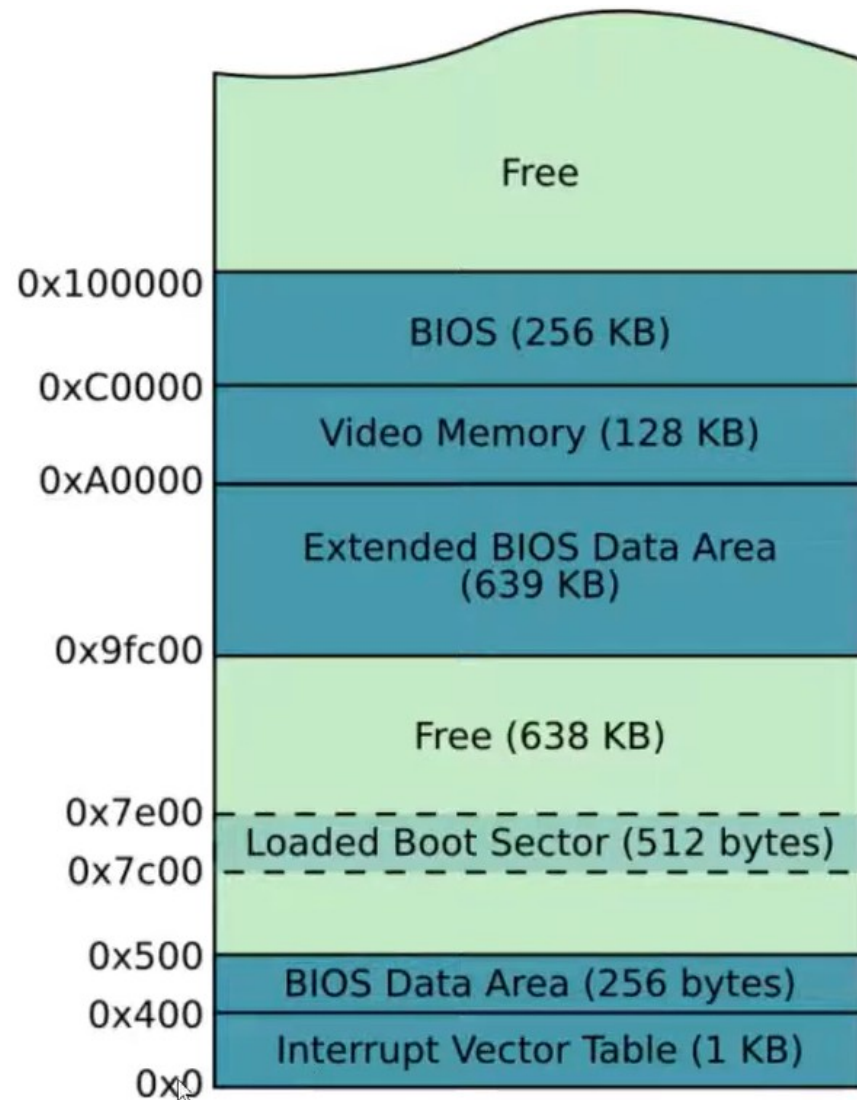
0xB0000 - 0xB7FFF (32 KB): Tipicamente usado para a memória de texto da placa de vídeo monocromática (MDA).

0xB8000 - 0xBFFFF (32 KB): Comumente usado para a memória de texto da placa de vídeo colorida (CGA/EGA/VGA). É aqui que os caracteres e seus atributos de cor são armazenados para exibição em modo texto.

Do endereço 0xC0000 até 0x100000 se encontra os **BIOS ROMs**, que contém o código do BIOS e de dispositivos como vídeo, rede, etc.

Endereços acima de 0x100000 não são acessíveis diretamente em modo real. Esta memória só pode ser usada em modo protegido (32 bits) ou long mode (64 bits).

Figura 4: Organização da memória durante o boot.



No projeto Fakenix é implementado um bootloader de 3 estágios:

Estágio 1: com 512 bytes, ocupando os endereços de 0x7C00 até 0x7DFF.

Estágio 2: com 1.024 bytes, ocupando os endereços de 0x7E00 até 0x81FF. Implementa o menu do sistema.

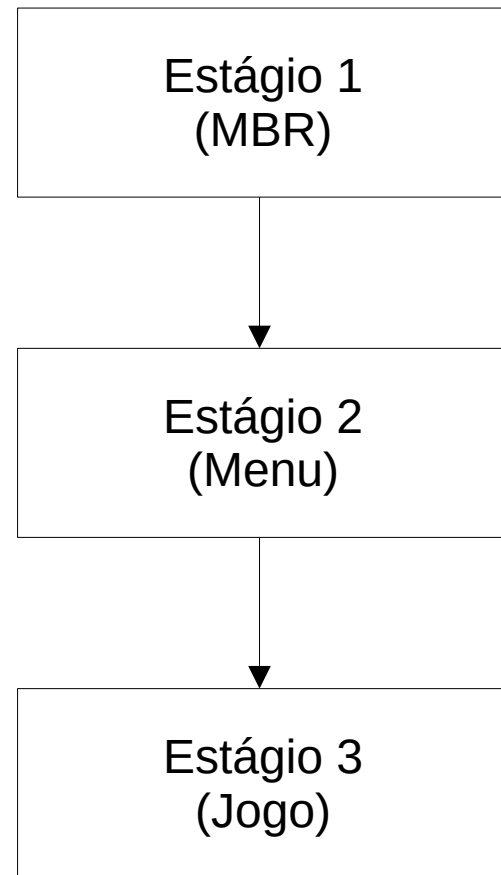
Estágio 3: com 2.048 bytes, ocupando os endereços de 0x8200 até 0x89FF. Implementa o jogo da cobrinha. Ele aloca ainda 1.536 bytes para o segmento de dados extras, nos endereços de 0x8A00 até 0x8FFF.

Adiante do estágio 3, se localiza a pilha do bootloader, que ocupa os endereços de 0x9000 até 0x18FFF. A pilha é compartilhada pelos 3 estágios.

O estágio 1 carrega na memória o estágio 2, que implementa o menu. O estágio 2 carrega o estágio 3, que num sistema prático seria o kernel do sistema operacional. Como não há um sistema operacional, o estágio 3 implementa o jogo da cobrinha, em modo real.

Figura 5: Estágios do bootloader Fakenix

O estágio 1 carrega na memória o estágio 2, que é o menu do sistema operacional. O estágio 2, por sua vez, carrega na memória o estágio 3, que é o jogo da cobrinha.



Obs.: Como não há sistema de arquivos, e será gerada uma imagem de disco no formato RAW, é importante que, se alterado o código dos estágios, se recalcule os endereços.